

School Management App

Layout Rendszer & Students CRUD Modul

React 18 + Laravel 12 API | 2026

1. A Layout rendszer felépítése

A Layout komponens az összes védett oldal közös 'váza' - Navbar felül, Sidebar oldalt, és a tényleges oldaltartalom középen. Ezt egyszer definiáljuk, és minden oldal automatikusan megkapja.

1.1 Layout.jsx – az Outlet szerepe

```
export default function Layout() {  
  return (  
    <div className='d-flex flex-column min-vh-100'>  
      <Navbar />  
      <div className='d-flex flex-grow-1'>  
        <Sidebar />  
        <main className='flex-grow-1 p-4 bg-light'>  
          <Outlet />  
        </main>  
      </div>  
    </div>  
  );  
}
```

Mi az az Outlet és miért kell?

Az Outlet a React Router speciális komponense - ide rendereli be az aktuális route-hoz tartozó oldalt (pl. DashboardPage, StudentListPage). Enélkül minden egyes oldalon újra kellene írni a Navbar-t és Sidebar-t. A beágyazott (nested) route-ok szülője a Layout, a gyerek route-ok tartalma az Outlet helyén jelenik meg.

1.2 Beágyazott route-ok az AppRoutes-ban

```
<Route element={(  
  <ProtectedRoute><Layout /></ProtectedRoute>  
>  
  <Route path='/dashboard' element={<DashboardPage />} />  
  <Route path='/students' element={<StudentListPage />} />  
>
```

```
<Route path='/students/create' element={<StudentFormPage />} />
<Route path='/students/:id/edit' element={<StudentFormPage />} />
</Route>
```

Miért csak EGYSZER van a ProtectedRoute?

A ProtectedRoute a SZÜLŐ route-ot védi, ami tartalmazza a Layout-ot. Mivel minden felsorolt gyerek route (dashboard, students stb.) a Layout Outlet-jén belül jelenik meg, mindegyik automatikusan védett lesz - nem kell minden egyes oldalt külön becsomagolni ProtectedRoute-tal.

2. Sidebar – szerepkör alapú menü szűrés

A Sidebar menüpontjai a bejelentkezett felhasználó szerepköre alapján jelennek meg vagy tűnnek el.

2.1 Deklaratív menü struktúra

```
const menuItems = [
  { to: '/dashboard', label: 'Áttekintés', icon: 'bi-speedometer2',
    roles: ['admin', 'teacher', 'parent'] },
  { to: '/teachers', label: 'Tanárok', icon: 'bi-person-badge',
    roles: ['admin'] },
  { to: '/fees', label: 'Díjak', icon: 'bi-cash-coin',
    roles: ['admin', 'parent'] },
  // ...
];

const visibleItems = menuItems.filter((item) =>
  item.roles.includes(user?.role)
);
```

Miért tömb + filter és nem sok if-else a JSX-ben?

Egy tömbben definiált struktúra DEKLARATÍV - egy pillantással látható az egész navigáció, könnyű új menüpontot hozzáadni (csak egy sor), és a szűrés logikája (filter) elkülönül a megjelenítéstől (map). Ha if-else-kkel írnánk meg JSX-ben, a kód gyorsan átláthatatlanná válna sok menüpontnál.

2.2 FONTOS: ez csak UI szűrés, nem valódi védelem

A Sidebar szűrés nem helyettesíti a backend védelmet!

Ha egy menüpont nem jelenik meg egy szülőnek, az CSAK azt jelenti hogy a felhasználói felületen nem látja a linket. Ha valaki MANUÁLISAN beírná az URL-t a böngészőbe (pl. /teachers), a Sidebar szűrés nem állítaná meg - a VALÓDI védelmet a backend route middleware (role:admin) és a Form Request authorize() metódusok biztosítják. A frontend szűrés csak jobb felhasználói élményért van, hogy ne mutassunk olyat amit úgysem tudna használni.

3. Dashboard – párhuzamos API hívások

A Dashboard oldal több különböző API végpontból gyűjt össze adatot egyetlen áttekintéshez.

3.1 Promise.all – párhuzamos lekérdezés

```
const [students, teachers, classes, overdueFees] = await Promise.all([
  api.get('/students', { params: { per_page: 1 } }),
  api.get('/teachers', { params: { per_page: 1 } }),
  api.get('/classes', { params: { per_page: 1 } }),
  api.get('/fees/overdue/list', { params: { per_page: 1 } }),
]);
```

Miért Promise.all() és nem egymás után await-elve?

Ha egymás után, await-tel hívnánk meg mind a 4 kérést, azok SORBAN futnának le - ha mindegyik 100ms-ig tart, összesen 400ms lenne az eredmény. A Promise.all() PÁRHUZAMOSAN indítja el mind a négyet, és megvárja amíg mindegyik befejeződik - így a teljes idő csak a lelassabb kérés idejével egyezik meg (kb. 100ms), nem az összes összegével.

Miért per_page: 1 a lekérdezéseknél?

A Dashboardnak nem kell a TELJES lista - csak a meta.total mező érdeklí (hány diák/tanár/osztály van összesen). A per_page: 1 minimalizálja az átvitt adatmennyiséget: csak 1 rekordot kér le ténylegesen, de a lapozási metaadat (total) így is tartalmazza a teljes darabszámot.

4. React Query Mutation minta – Students CRUD

A useStudents.js hook fájl tartalmazza az összes React Query logikát a diák modulhoz - ez a minta ismétlődik majd minden más modulnál (Teachers, Classes stb.).

4.1 Query vs Mutation - a különbség

	useQuery	useMutation
Mikor fut le	Automatikusan, komponens megjelenésekor	Csak amikor explicit meghívjuk (mutate())
HTTP metódus	GET (adat olvasás)	POST/PUT/DELETE (adat módosítás)
Cache-elés	Igen, automatikus	Nem - de invalidálhatja a query cache-t
Projektben	useStudents(), useStudent(id)	useCreateStudent(), useUpdateStudent(), useDeleteStudent()

4.2 Lista lekérdezés – keepPreviousData

```
export function useStudents(page = 1) {
  return useQuery({
    queryKey: ['students', page],
    queryFn: () => api.get('/students', { params: { page } })
      .then((res) => res.data),
    keepPreviousData: true,
  });
}
```

Miért keepPreviousData: true?

Lapozáskor (2. oldalra kattintva) enélkül a táblázat egy pillanatra üressé válna (loading állapot), amíg az új oldal adata megérkezik - ez zavaró 'villogást' okozna. A keepPreviousData megtartja a RÉGI adatot látható állapotban amíg az ÚJ meg nem érkezik, így a felhasználó folyamatosan lát valamit a képernyőn.

4.3 Egyetlen elem lekérdezése – enabled feltétel

```
export function useStudent(id) {
  return useQuery({
    queryKey: ['students', id],
    queryFn: () => api.get(`/students/${id}`).then((res) =>
      res.data.data),
    enabled: !!id,
  });
}
```

Miért enabled: !!id?

A StudentFormPage komponens KÉT módban működik: létrehozás (nincs id az URL-ben) és szerkesztés (van id). Létrehozás módban id === undefined - ilyenkor NEM szabad lefuttatni a lekérdezést, mert a GET /students/undefined hibás kérés lenne. Az enabled: !!id (dupla tagadás - undefined-ből false-t csinál) feltételesen ki/bekapcsolja a query futását.

4.4 Mutation és cache invalidálás

```
export function useCreateStudent() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (data) => api.post('/students', data).then((res) => res.data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['students'] });
    },
  });
}
```

Mit csinál az invalidateQueries()?

Sikeres létrehozás után 'elavultnak' jelöli az összes ['students'] kulcsú cache bejegyzést. A React Query ennek hatására automatikusan ÚJRA lekéri a listát a szervertől - így a StudentListPage azonnal frissül és megjeleníti az új diákokat, anélkül hogy manuálisan újra kellene tölteni az oldalt vagy kézzel frissíteni a state-et.

5. StudentFormPage – egy komponens, két mód

Ahelyett hogy külön CreateStudentPage és EditStudentPage komponenst írnánk (ami sok duplikált kódhoz vezetne), egyetlen StudentFormPage komponens szolgálja ki mindkét esetet.

5.1 A mód eldöntése az URL alapján

```
const { id } = useParams();
const isEditMode = Boolean(id);

// /students/create      -> id === undefined -> isEditMode === false
// /students/5/edit      -> id === '5'         -> isEditMode === true
```

5.2 Form feltöltése szerkesztéskor – reset() vs defaultValues

```
useEffect(() => {
  if (isEditMode && student) {
    reset({
      name: student.name,
      email: student.email,
      class_id: student.school_class?.id ?? '',
    });
  }
}, [isEditMode, student, reset]);
```

Miért reset() a useEffect-ben és nem defaultValues a useForm()-ban?

A useForm() AZONNAL lefut a komponens megjelenésekor - ekkor a diák adata (student) még nem érkezett meg a szerverről (aszinkron API hívás). Ha defaultValues-t használnánk, az még üres objektumot látna. A reset() viszont bármikor meghívható és FRISSÍTÍ a form mezőit - ezt a useEffect-ben hívjuk meg, amikor a student adat ténylegesen megérkezik (a függőségi tömbben szerepel).

5.3 mutateAsync – megvárható mutation

```
if (isEditMode) {
  await updateStudent.mutateAsync(data);
} else {
  await createStudent.mutateAsync(data);
}

navigate('/students');
```

mutate() vs mutateAsync() - a különbség

A sima mutate() 'tűzz és felejtse el' jellegű - nem ad vissza Promise-t, nem await-elhető. A mutateAsync() Promise-t ad vissza, amit MEG KELL VÁRNI (await) - ez kell nekünk, mert csak a MENTÉS BEFEJEZŐDÉSE UTÁN akarjuk átirányítani a felhasználót a listára (navigate('/students')). Ha mutate()-et használnánk, az átirányítás azonnal megtörténne, még mielőtt a szerver válaszolt volna.

5.4 Validációs hibák kinyerése a Laravel válaszból

```
const validationErrors = error.response?.data?.errors;
const firstError = validationErrors
  ? Object.values(validationErrors)[0][0]
  : null;

setServerError(
```

```
firstError || error.response?.data?.message || 'Hiba történt a mentés során.'
);
```

Miért `Object.values(...)[0][0]`?

A Laravel 422-es válasza így néz ki: { errors: { email: ['Ez az email már foglalt.'], name: ['A név kötelező.'] } }. Az errors egy OBJEKTUM, ahol minden kulcs (mezőnév) egy TÖMBÖT tartalmaz (mert egy mezőnek több hibája is lehet egyszerre). Az `Object.values(errors)` kinyeri az összes tömböt egy tömbbe, a `[0]` az ELSŐ mező hibáit adja, a `[0][0]` pedig az első mező ELSŐ hibaüzenetét - ez egy egyszerű, konkrét visszajelzés a felhasználónak.

6. Hibakeresési napló – Students modul

Hiba	Ok	Megoldás
Nem tölthető ki a form (class_id)	Nem volt még osztály az adatbázisban	SchoolClassSeeder létrehozása teszt osztályokkal
'The roll number field is required'	roll_number véletlenül bekerült a rules()-ba	Eltávolítás - a Service generálja automatikusan
gender validáció sosem passzol	Elgépelés: 'femala' a rules()-ban	Javítás 'female'-re

Az általános tanulság: auto-generált mezők NEM lehetnek kötelezőek a Request-ben

Ugyanaz a minta mint a vizsgaeredmény grade/status vagy a fee receipt_number mezőknél: ha egy mező értékét a BACKEND generálja (Service rétegben), azt SOHA nem szabad required (vagy egyáltalán jelenlévő) szabályként felvenni a Form Request rules()-ába - mert a frontend nem küldi, a validáció mindig el fogja utasítani a kérést.

7. Frontend mappa struktúra – jelenlegi állapot

```
resources/js/
  api/axios.js
  context/AuthContext.jsx
  hooks/
    useAuth.js
    useStudents.js      - list, single, create, update, delete
    useClasses.js       - dropdown adatforrás
    useDashboardStats.js - Promise.all összesítés
  components/layout/
    Layout.jsx          - Navbar + Sidebar + Outlet váz
    Navbar.jsx          - user info, kijelentkezés
```

```
Sidebar.jsx          - szerepkör alapú menü szűrés
pages/
  LoginPage.jsx
  DashboardPage.jsx  - statisztika kártyák
  students/
    StudentListPage.jsx - táblázat, lapozás, törlés
    StudentFormPage.jsx - create ÉS edit egy komponensben
routes/AppRoutes.jsx - beágyazott route-ok Layout-tal
app.jsx
```

8. Ismétlődő minta – ez lesz minden modul váza

A Students modulnál kialakított minta közvetlenül átvihető Teachers, Classes, Subjects és a többi modulra:

1. hooks/useXxx.js
 - useXxx(page) -> lista lekérdezés lapozással
 - useXxxItem(id) -> egy elem lekérdezés (enabled: !!id)
 - useCreateXxx() -> mutation + invalidateQueries
 - useUpdateXxx(id) -> mutation + invalidateQueries
 - useDeleteXxx() -> mutation + invalidateQueries
2. pages/xxx/XxxListPage.jsx
 - useXxx() hívás, táblázat, lapozás, törlés gomb + confirm
3. pages/xxx/XxxFormPage.jsx
 - useParams() -> isEditMode
 - useForm() + reset() useEffect-ben szerkesztéshez
 - mutateAsync() + navigate() sikeres mentés után
 - szervertől hiba kinyerése és megjelenítése
4. routes/AppRoutes.jsx bővítése a 3 route-tal (list, create, edit)

– School Management App fejlesztési dokumentáció –